Databases & SQL

2026-05-12 · plucky rhubarb Hase

In Comparison, Dumbledore's Pensieve is a joke.

The Why

We have a service that classifies, but it does not remember anything. The moment the response leaves the server, the prediction is gone. This makes it hard to track the model's performance over time, compare versions, or build any type of features and functionality that revolve around past predictions.

We need to wire the API onto a database. This block is the first that adds persistent state. Every later block depends on the database schema you design today.

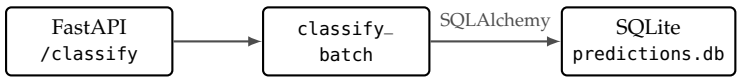

Figure 1: The persistence flow. `POST /classify` computes a prediction with `classify_batch`, the handler maps the result through the `Prediction` SQLAlchemy model, and a row lands in the `predictions` table inside the `predictions.db` SQLite file before the response goes out.

Why persist at all? A response that vanishes the moment it is sent is a response that cannot be counted, compared, or queried. A database makes the prediction permanent: Every drawing becomes a row, the class label becomes a column, the confidence becomes another, and the model version becomes a third.

Beyond this course, the same table is where production teams build retraining datasets, let users correct wrong predictions for labelled data, and detect distribution drift. We do not do any of those here, but it is trivial to see how every one of them starts with having the database.

Hands On Experience

CONSIDER THIS SCENARIO: A teammate asks how confident the model has been on average across the last hundred predictions. You realise nobody can answer that question. The API returned the confidence to every caller, but nothing recorded it. The only way back to those numbers is to ask the users to draw their digits again, which is not an option. In an increasingly data-driven world, data defines your product, your moat, and your competitive advantage. Not having the data is like throwing away your most valuable asset, and until someone comes up with a solution for time-travel, it's a non-recoverable loss.

THE SCHEMA IS A CONTRACT, the same way the API contract is a contract. Get it right and every later question, distribution per digit, average confidence per model version, error rate over time, becomes a query. Get it wrong and you find out later, with a hundred thousand rows already written, that the column you need does not exist. Schemas evolve through migrations, which we cover at the end of the block as an optional path.

DESIGNING SCHEMAS is its own discipline. We design the `predictions` table once and keep it that way for the mainline of the course. The optional Alembic exercise at the end lets you feel the cycle of design, deploy, regret, migrate on a real database.

THIS SIXTH BLOCK introduces persistence for PixelWise. By the end you will have

- written your first SQL queries against a SQLite database,
- modelled the `predictions` table as a SQLAlchemy class,
- wired `POST /classify` so every prediction becomes a row,
- replaced the `GET /results` stub with a real query,
- configured database credentials through `.env`,
- and, optionally, run your first schema migration with Alembic.

Optional Exercise: Catch Up to `v0.4`

CATCHING UP VIA TAG. This catch-up extends the one in Block 5. If you have not done that one, do it first: It lands you at `v0.3`, the end-of-Block 4 state. The exercise here picks up from there and rewinds to `v0.4`, the end-of-Block 5 state, with the FastAPI service exposing `POST /classify` and `GET /health`, API key authentication, basic rate limiting, and a `systemd`-supervised `uvicorn` running on `prod`.

TAG MAP. `v0.4` marks the end of Block 5, `v0.5` will mark the end of this block. Browse the full set at `https://github.com/schutera/pixelwise/tags`.

BOTH VMS THIS TIME. Block 5 ended with the service actually running on `prod`, so unlike the previous catch-up, this one brings both machines up to `v0.4`. The Git-tracked changes since `v0.3` include `app/main.py`, `deploy/pixelwise.service`, an extended `setup-server.sh` that installs the systemd unit on `prod`, an updated `requirements.txt` adding `fastapi`, `uvicorn`, `slowapi`, and `pydantic`, and a refreshed `.env.example` with `SECRET_API_KEY`. The local `.env` on `prod` needs the new key written by hand; `setup-server.sh` then pulls the model, copies the unit, enables, and restarts the service in one shot.

RUN THE CATCH-UP ON `dev`. Rewind the repository to `v0.4`, refresh `.env` from the updated example and edit it to set `SECRET_API_KEY`, re-install pinned dependencies if your virtual environment was wiped, then run `setup-server.sh` to pull the model artefact. The exact commands are in the margin.

SEQUENCE ON `dev`.

```
cd ~/pixelwise
git fetch origin --tags
git reset --hard v0.4
cp .env.example .env
nano .env
source .venv/bin/activate
pip install -r requirements.txt
bash setup-server.sh
```

Edit `.env` to set `SECRET_API_KEY` to any string of your choosing; `openssl rand -hex 32` prints a clean random value if you want one. If you already have a `.env` from earlier work, skip the `cp` and just edit the file in place. If `.venv/` is missing, run `python3 -m venv .venv` before the `source` step.

RUN THE CATCH-UP ON `prod`. Same sequence on `prod`, where `setup-server.sh` also copies the systemd unit into place and restarts the service. After the script finishes, verify the API is reachable from `dev`.

SEQUENCE ON `prod`.

```
ssh produser@192.168.56.11
cd /opt/pixelwise
git fetch origin --tags
git reset --hard v0.4
cp .env.example .env
nano .env
source .venv/bin/activate
pip install -r requirements.txt
bash setup-server.sh
```

Use the same `SECRET_API_KEY` value as on `dev`. If you already have a `.env` on `prod`, skip the `cp` and edit the file in place. Then from `dev`: `curl http://192.168.56.11:8000/health` should return `{"status":"ok"...}`.

YOU ARE READY PLAYER ONE.

Relational Databases and SQL

A RELATIONAL DATABASE STORES DATA IN TABLES. Each table has a fixed schema: a list of named columns, each with a type. Each row is a single record. The schema is the contract that every record must follow; the database refuses inserts or queries that violate it.

STRUCTURED QUERY LANGUAGE (SQL) is the language you use to talk to a relational database. The database is a single file, there is no server to install, no users, no network configuration. For a single-developer prototype or a desktop application, this is exactly what you want. Four operations cover almost everything:

```
-- Insert a row
INSERT INTO predictions (prediction, confidence,
                         model_version)
VALUES ('7', 0.94, 'v1');

-- Read rows
SELECT prediction, confidence FROM predictions
    WHERE model_version = 'v1'
    ORDER BY created_at DESC
    LIMIT 10;

-- Delete a row
DELETE FROM predictions WHERE id = 42;
```

VOCABULARY TO KEEP STRAIGHT. A table is a named set of rows; a row is one record; a column is a named field with a type; a primary key is the column or columns that uniquely identify each row. Most tables use an auto-incrementing integer or a UUID as their key.

id	prediction	confidence	model_version
1	7	0.94	v1
2	3	0.81	v1
3	5	0.99	v1

WHAT YOU WILL MEET LATER.

- `JOIN` combines rows from multiple tables.
- `UPDATE` adjusts existing values without changing the schema.
- `GROUP BY`, `COUNT`, and `AVG` aggregate.
- `ORDER BY` and `LIMIT` shape the output.
- `CREATE INDEX` speeds up queries that scan large tables.

The full treatment belongs in a dedicated database course; the four operations above are enough to get PixelWise running.

PostgreSQL for Production

POSTGRESQL IS A PRODUCTION DATABASE. It runs as a separate process, accepts connections over the network, has real users with real permissions, and supports concurrent reads and writes without serialising the whole file. `psql` is the command line client.

WHY A DEDICATED USER? LEAST PRIVILEGE The `postgres` superuser can drop any database, create any user, and read any table. A dedicated `pixelwise` user can only do what its privileges allow, which by default is read and write its own tables. The principle is the same as the non-root `produser` on your server. You provision a `pixelwise` role and a `pixelwise` database owned by that role inside PostgreSQL, then verify the credentials with `psql` before any Python is involved.

WHERE SQLITE STOPS SHORT? SQLite has no user authentication, no password protection, and limited concurrency - due to file locking.

PostgreSQL: `https://www.postgresql.org/`
SQLite: `https://www.sqlite.org/`

Exercise: SQLite on the Server

While you could set things up manually on `prod`, it is a best practice to automate every step of the server configuration through `setup-server.sh`; you edit it on `dev`, commit, push, and `bash setup-server.sh` carries the work on both VMs. The one manual step is writing the secret `DB_PASSWORD` into the local `.env` on each VM, since secrets never travel through git. Provisioning on `dev` too means you can write `models.py`, run `init_db.py`, and inspect rows with `psql` without ever ssh'ing to `prod`.

Extend `setup-server.sh` on `dev`. Add `postgresql` to the `apt install` line, then append a stanza that provisions the `pixelwise` role and database wherever `.env` is present:

```
cd ~/pixelwise
nano setup-server.sh
```

Resolve the script directory at the top. Add this line to `setup-server.sh` right after the shebang, so every later stanza can locate the repo root regardless of the caller's working directory:

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" \
  && pwd)"
```

Append the provisioning stanza after the systemd block:

```
# Provision the pixelwise role and database on every VM
if command -v psql >/dev/null 2>&1 && \
   [ -f "$SCRIPT_DIR/.env" ]; then
    set -a; source "$SCRIPT_DIR/.env"; set +a
    sudo -u postgres psql -tAc \
        "SELECT 1 FROM pg_roles WHERE rolname='pixelwise'" \
        | grep -q 1 || \
    sudo -u postgres psql -c \
        "CREATE USER pixelwise \
         WITH PASSWORD '$DB_PASSWORD';"
    sudo -u postgres psql -tAc \
        "SELECT 1 FROM pg_database WHERE datname='pixelwise'" \
        | grep -q 1 || \
    sudo -u postgres createdb -O pixelwise pixelwise
fi
```

The connection string is a single URL: `sqlite:////absolute/path/to/file.db`. Note the four slashes after the colon: three for the URL scheme, one for the absolute path. A relative path uses three slashes (`sqlite:///./pixelwise.db`). The application reads this from `.env`, so the same code works on `dev` and `prod` with different paths.

Idempotent by design. Each `psql -tAc` probe asks PostgreSQL whether the role or database already exists; the `||` short-circuits the create when the answer is yes. Run the script ten times, the role and database are still there exactly once. `SCRIPT_DIR` resolves to whichever checkout invoked the script, so the same stanza runs on `dev` and `prod`; the systemd block from Block 5 stays gated behind `id produser` since the service only ever runs on `prod`.

Document the new secret in `.env.example` on `dev`, then commit and push:

```
echo "DB_PASSWORD=" >> .env.example
git add setup-server.sh .env.example
git commit -m "Provision PostgreSQL"
git push
```

Apply on `dev` first. Write the password into your local `.env` and run the script:

```
echo "DB_PASSWORD=database" >> .env
bash setup-server.sh
```

Repeat on `prod`. Same shape, plus the `git pull` to bring in the committed script:

```
ssh produser@192.168.56.11
cd /opt/pixelwise
git pull
echo "DB_PASSWORD=database" >> .env
bash setup-server.sh
```

The first run on each VM installs `postgresql` and provisions the `pixelwise` role and database; subsequent runs change nothing because the provisioning stanza is idempotent.

Verify the connection on `prod` with the application user:

```
psql -U pixelwise -d pixelwise -h localhost
```

`pixelwise=>` prompt means the role and database are wired up. If `psql` fails to connect, fix the credentials before going any further. A working `psql` is the foundation of everything the rest of the block builds on. If it worked you know

Did it take on `dev`?

```
psql -U pixelwise -d pixelwise -h localhost
```

Enter the password from `.env`. A `pixelwise=>` prompt means the role exists, the database is yours, and the password matches. If the connection is refused, re-read the script output before moving on to `prod`.

Wipe and start fresh. The provisioning stanza skips creation when the role or database already exists, so editing `DB_PASSWORD` after a first run does not propagate. If `psql` rejects the password or reports a missing database, drop both and re-run:

```
sudo -u postgres dropdb pixelwise
sudo -u postgres dropuser pixelwise
bash setup-server.sh
```

What each flag does.

- `-U pixelwise` selects the role to log in as.
- `-d pixelwise` selects the database to connect to.
- `-h localhost` forces a TCP connection to the local machine, which makes `psql` prompt for the password from `.env` instead of trusting the Unix socket's peer authentication on the `postgres` system user.

Role and database share a name. The setup script creates both:

- A role `pixelwise` with a password.
- A database `pixelwise` owned by that role.

PostgreSQL keeps roles and databases as independent objects, so the matching names are a readability convention, not a requirement; the application could just as well live in a database called `predictions` owned by a role called `api`.

SQLAlchemy: Tables as Python Classes

SQLAlchemy is a Python library that maps database tables to Python classes. You define a `Prediction` class with fields like `prediction`, `model_version`, and `created_at`; SQLAlchemy generates the SQL to create the table, insert rows, and query data. This is called an ORM, an object-relational mapper, and is standard in Python web development.

SQLAlchemy:
https://www.sqlalchemy.org/
SQLAlchemy Tutorial:
https://docs.sqlalchemy.org/en/20/tutorial/

Python is ergonomics. Two reasons, why choosing to use an ORM: Writing Python is faster than writing string SQL, and the editor knows the field names. The second is safety: SQLAlchemy uses parameterised queries by default, so user input never ends up concatenated into a SQL statement. SQL injection becomes a footgun the framework refuses to hand you.

An engine and a session are the runtime objects that connect the model to the database. `create_engine` reads the `DATABASE_URL` from the environment and opens a pool of connections; `sessionmaker` returns a `SessionLocal` factory the handlers call once per request; `Base.metadata.create_all(engine)` creates the table on first run.

The connection string is a single URL: `postgresql://user:pass@host/dbname`. It encodes everything the driver needs to reach the database, and it lives in `.env` for access restrictions and security. A leaked `DATABASE_URL` is a leaked database.

Exercise: The `Prediction` Model

INSTALL THE DRIVER AND THE ORM ON `dev` into the virtual environment, then pin the new packages to `requirements.txt` so `prod` picks them up on the next deploy.

INSTALL.

```
cd ~/pixelwise
source .venv/bin/activate
pip install sqlalchemy
psycopg2-binary
pip freeze > requirements.txt
```

WRITE `app/models.py` on `dev` with the schema for one prediction, the class label, the confidence, and the model version:

```
from sqlalchemy import (Column, Integer, String,
                        Float, DateTime)
from sqlalchemy.orm import declarative_base
from datetime import datetime

Base = declarative_base()

class Prediction(Base):
    __tablename__ = "predictions"
    id = Column(Integer, primary_key=True)
    prediction = Column(String, nullable=False)
    confidence = Column(Float, nullable=False)
    model_version = Column(String, nullable=False)
    created_at = Column(DateTime,
                        default=datetime.utcnow)
```

THREE COLUMNS PLUS BOOKKEEPING. `prediction`, `confidence`, and `model_version` carry the answer; `id` and `created_at` are bookkeeping the database fills in for you. Every later question, distribution per digit, average confidence per model version, error rate over time, lives in this table.

DOCUMENT `DATABASE_URL` in `~/pixelwise/.env.example` on `dev`, and add the same line to your local `.env` on both VMs so the engine can find the database. The URL interpolates `DB_PASSWORD` from the same file, so the secret lives in one place:

```
DATABASE_URL=postgresql://pixelwise:${DB_PASSWORD}@localhost/pixelwise
```

CREATE THE TABLE ONCE from a small initialisation script saved as `init_db.py` at the repository root on `dev`. The script reads `DATABASE_URL` you set in the previous exercise and calls `Base.metadata.create_all` to materialise every model class into a table:

```
from sqlalchemy import create_engine
from app.models import Base
import os
from dotenv import load_dotenv

load_dotenv()
engine = create_engine(os.getenv("DATABASE_URL"))
Base.metadata.create_all(engine)
```

COMMIT AND DEPLOY. Stage the new and changed files on `dev`, push, and on each VM install the pinned dependencies and run `python init_db.py` to create the schema. Then confirm with `psql` that the `predictions` table is listed:

```
psql -U pixelwise -d pixelwise -h localhost -c "\dt"
```

INITIALISE THE SCHEMA. by running `python init_db.py` on `dev`.

COMMIT AND DEPLOY.

On `dev`:

```
git add app/models.py init_db.py
  requirements.txt .env.example
git commit -m "Add Prediction
model"
git push
source .venv/bin/activate
python init_db.py
```

On `prod`:

```
cd /opt/pixelwise && git pull
source .venv/bin/activate
pip install -r requirements.txt
python init_db.py
```

OPTIONAL: ONE-SHOT DEPLOY. Make `bash setup-server.sh` also install pinned dependencies and create the schema by appending two stanzas after the Postgres provisioning block:

```
if [ -d "$SCRIPT_DIR/.venv" ] &&
  [ -f
"$SCRIPT_DIR/requirements.txt"
]; then
  source
"$SCRIPT_DIR/.venv/bin/activate"
  pip install -r
    "$SCRIPT_DIR/requirements.txt"
fi
```

```
if [ -f "$SCRIPT_DIR/init_db.py" ];
then
  (cd "$SCRIPT_DIR" && python
init_db.py)
fi
```

Keep the order: The init stanza relies on the venv the first stanza activates.

Exercise: Wiring the API to the Database

The database is ready. Let's make the API populate it with real data.

Edit `app/main.py` on `dev` so `POST /classify` writes a row before returning the response, and `GET /results` returns real rows instead of the current scaffold. The change lands in three pieces: Two new imports at the top of the file, an addition to the classify handler, and a real body for results.

```
from app.models import Prediction, SessionLocal
```

Two new imports. `Prediction` is the row class you defined in `app/models.py`; `SessionLocal` is the session factory built from the engine. Add the line at the top of `main.py` next to the existing imports.

```
@app.post("/classify",
          response_model=ClassifyResponse,
          dependencies=[Depends(verify_api_key)])
@limiter.limit("30/minute")
def classify(request: Request,
             req: ClassifyRequest):
    arr = np.array(req.pixels,
                   dtype=np.uint8)[np.newaxis]
    result = classify_batch(arr)[0]

    db = SessionLocal()
    db.add(Prediction(
        prediction=result["prediction"],
        confidence=result["confidence"],
        model_version="v1"))
    db.commit()
    db.close()

    return result
```

Classify now writes a row. The decorators and signature from Block 5 stay untouched. The body picks up one new intermediate, `result`, so the same dict can be persisted and returned, then a five-line session block that opens a `SessionLocal`, adds the new `Prediction`, commits, and closes.

A session is the unit of database work: Open it at the start of the handler, close it at the end. FastAPI documents a more elegant variant through `Depends`; the manual version here is the simplest one that shows what is happening. The returned `result` dict is unchanged, so the response body stays identical to the contract we defined in Block 5.

```
@app.get("/results")
def results():
    db = SessionLocal()
    rows = (db.query(Prediction)
              .order_by(Prediction.created_at.desc())
              .limit(20).all())
    db.close()
    return {"results": [
        {"id": r.id,
         "prediction": r.prediction,
```

Results runs one query. It pulls the twenty newest rows from `predictions`, ordered by `created_at` descending, and shapes each row into the JSON object the frontend will expect in the next block. The Block 5 stub that returned an empty list is gone.

```
         "confidence": r.confidence,
         "model_version": r.model_version,
         "created_at": r.created_at.isoformat()}
        for r in rows]}
```

Restart the service. On `dev` the running `uvicorn --reload` picks up the new code on save. On `prod` restart through systemd so the unit re-reads `app/main.py`:

```
sudo systemctl restart pixelwise   # prod only
```

Smoke test the wiring with one unauthenticated `GET /results` call. An empty table is exactly what you want here: A successful response with an empty list proves the engine opened, the session works, the query ran, and the JSON shape is right. Any of those broken and the call returns a 500, not an empty list:

```
# on dev, against the local uvicorn
curl -s http://localhost:8000/results \
  | python3 -m json.tool

# on dev, against prod via the API gateway
curl -s http://192.168.56.11:8000/results \
  | python3 -m json.tool
```

Commit and tag `v0.5`.

```
git add app/main.py app/models.py
  init_db.py requirements.txt
  .env.example
git commit -m "Add PostgreSQL
persistence"
git tag v0.5
git push --follow-tags
```

Stage on `dev`, commit with a message that names the change, tag the result, and push both the commit and the tag in one shot.

Optional: Feed it payload. So far our flow has never been hit by real digit drawings, and on that note testing really came short. Build a 28-by-28 zero payload once, perhaps wrapped in a `probe_init.py`, then use it to test the API and the database.

Optional Exercise: First Migration with Alembic

SCHEMAS ALWAYS CHANGE. You design the table, deploy it, store 500 predictions. Then you realise you forgot a column. You cannot drop the table and recreate it; those 500 rows are real data. You need a tool that alters the live schema without losing them. This exercise is optional: The mainline of the course does not depend on it, but running it once gives you the feel of the migration cycle on a real database. The example below adds a `client_ip` column for debugging, but any new column works.

Alembic: `https://alembic.sqlalchemy.org/`

INITIALISE ALEMBIC ON DEV. Alembic is the migration tool for SQLAlchemy; the cycle is `init`, `revision --autogenerate`, `upgrade head`, run in that order. All file edits in this exercise happen on `dev`. `alembic init` writes its scaffold into the current directory, so run everything from the repository root:

THE METAPHOR. A migration is a recipe that describes a change to the schema and how to roll it back. The migration tool keeps an internal log of which migrations have run, applies new ones in order, and refuses to run the same one twice. The result is that the schema in the database always matches the migrations folder in Git.

```
cd ~/pixelwise
source .venv/bin/activate
pip install alembic
pip freeze > requirements.txt
alembic init alembic
```

The `pip freeze` step captures the new package in `requirements.txt` so `prod` picks it up on the next deploy.

Edit `alembic.ini` so `sqlalchemy.url` reads from `.env`, and edit `alembic/env.py` to import `Base` from `app.models` so autogenerate sees your tables.

READ THE GENERATED CONFIG. `alembic init alembic` writes a folder of starter files. Skim them once before you change anything; the structure is small and the mental model becomes obvious by reading the templates.

ADD A NEW COLUMN to the model. In `~/pixelwise/app/models.py` on `dev`, add `client_ip = Column(String)` between the `confidence` and `model_version` lines.

GENERATE THE MIGRATION from the repository root on `dev`:

```
cd ~/pixelwise
alembic revision --autogenerate \
    -m "add client_ip column"
```

READ THE MIGRATION BEFORE YOU RUN IT. The generated script lives in `alembic/versions/`. Open it, confirm the only operation is `add_column("client_ip")`, and only then run `upgrade head`. A wrong migration on a populated table is much harder to undo than to prevent.

WIRE `client_ip` INTO THE HANDLER in `~/pixelwise/app/main.py` on `dev`. `Request` is already imported and `request: Request` is already a parameter on `classify` from the Block 5 rate limiter, so the only change is one keyword argument in the `Prediction(...)` call:

```
db.add(Prediction(
    prediction=result["prediction"],
    confidence=result["confidence"],
    client_ip=request.client.host,
    model_version="v1"))
```

COMMIT AND DEPLOY. On `prod`, order matters: Install the new pinned dependency and apply the migration first, then run `setup-server.sh`. `setup-server.sh` restarts the service with code that references the new column, so if the migration has not run yet, the handler will try to INSERT a column that does not exist in the database. From `dev`, stage the changes and push:

```
cd ~/pixelwise
git add app/main.py app/models.py alembic/ \
    alembic.ini requirements.txt
git commit -m "Add client_ip column via Alembic"
git push
```

Then on `prod`, pull, install the new dependency, run the migration, and only then restart the service:

```
cd /opt/pixelwise
git pull
source .venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
bash setup-server.sh
```

VERIFY ON `prod` WITH `psql`:

```
psql -U pixelwise -d pixelwise -h localhost \
  -c "SELECT prediction, client_ip
      FROM predictions
      ORDER BY created_at DESC LIMIT 5;"
```

Old rows show `NULL` for `client_ip`; new predictions store the caller's address now that the field is wired in. That gap is the visible signature of a migration: The column exists everywhere, but its values populate only from the moment the new code shipped.

Self-Reflection and Recap

Self-Reflection questions to guide your thoughts during and after the exercises:

- Why do we store predictions in a database rather than just returning them in the API response?
- What is the difference between dropping and recreating a table versus running a migration?
- If you ran the optional Alembic exercise, why do old rows show `NULL` for the new column while later rows carry a value?
- What happens if you store the database password directly in your Python code instead of in `.env`?
- Why is a dedicated database user with minimal privileges better than connecting as the superuser?
- How does the ORM make SQL injection harder, and where does the protection break down?
- Why does it pay to think the schema through up front rather than adding columns later?
- If you did the optional Alembic section: what is the difference between dropping and recreating a table versus running a migration, and why did the new column show `NULL` on every existing row?

Recap of key concepts:

- Relational databases store data in typed tables; SQL is the language that queries and changes them.
- SQLite is the database for this course: a single file, no server, no users, no passwords. PostgreSQL is the production next step we stop short of here.
- SQLAlchemy maps tables to Python classes and uses parameterised queries by default, so the model code looks the same against either backend.
- `DATABASE_URL` lives in `.env` so each host can point at a different file without code changes.
- Optionally, Alembic generates and applies migrations so schemas can evolve without losing rows.

Teaser. We classify, we store, we query, but still no user can touch our backend.

BRIDGE. The service classifies, stores, and queries, but the only way in is `curl` with an API key on a non-standard port. No browser, no URL, no human-friendly surface. This needs to change next we put `nginx` in front of the FastAPI process, serves a small drawing frontend over plain HTTP and HTTPS, and route browser traffic into the same `POST /classify` and `GET /results` handlers you just wired up. The database stays where it is; the prediction it stores will start arriving from a canvas in a browser rather than a shell command.